



RAPPORT SUR LA GESTION D'UNE BIBLIOTHEQUE

Option : Système d'Information et Réseaux

**THEME : Conception et Réalisation d'un Système Intégré de Gestion de
Bibliothèque (SIGB)**

MEMBRES DU GROUPE :

- ✓ OUATTARA KONIBA W.AUGUSTE ELVIS
- ✓ KOUSSOUBE DRISSA

PROFESSEUR :
Mr ILBOUDO A

Année académique 2025-2026

SOMMAIRE

SOMMAIRE.....	I
DEDICACE	III
REMERCIEMENTS.....	IV
RESUME	V
LISTE DES SIGLES ET ABREVIATIONS.....	VI
LISTE DES FIGURES	VII
INTRODUCTION GENERALE	1
CHAPITRE I : STRUCTURE D’ACCUEIL	2
I. Présentation de l’institution.....	2
II. Justificatif du choix de la structure d’accueil.....	2
III. Objectifs du futur système	2
CHAPITRE II : ANALYSE ET SPÉCIFICATIONS UML	3
I. Analyse fonctionnelle	3
II. Modélisation dynamique.....	10
III. Modélisation des états (Les 5 cycles de vie).....	18
IV. Interactions et Structure Statique	22
• Vue d'ensemble : Il montre comment le module Utilisateur communique avec le module Emprunt via des interfaces de service.....	22
CHAPITRE III : CONCEPTION TECHNIQUE.....	27
I. Architecture logicielle	27
II. Composants et Navigation	28
• Composants Backend : Ici nous avons une séparation stricte entre les API Controllers, les Services métier et les Data Repositories.....	28
III. Modèle de la Base de Données(Schéma Relationnel)	VII
IV. Justification des Choix Technologiques.....	VIII
V. Documentation des API REST	VIII
CHAPITRE IV : REALISATION ET IMPLEMENTATION.....	IX
I. Environnement de Développement et Technologies Utilisées	IX

II. Description des Modules Clés.....	X
III. Présentation de la Solution pour les Tests.....	XI
IV. Bilan du projet.....	XIV
CONCLUSION GENERALE ET PERSPECTIVES	XVI
I. Bilan du projet	XVI
II. Bilan du projet.....	XVI
III. Perspectives d'Evolution	XVII

DEDICACE

Nous dédions ce modeste travail :

- À nos parents, pour leurs sacrifices inconditionnels et leur soutien indéfectible tout au long de notre parcours scolaire.
 - À nos professeurs de l'ESTA, pour la qualité de l'enseignement et leur disponibilité.
- À tous les étudiants de la filière SIR, en souvenir de nos moments d'apprentissage collectif.

REMERCIEMENTS

Avant tout développement sur ce projet, nous tenons à remercier le Seigneur Tout-Puissant pour nous avoir donné la force et la santé nécessaires pour mener à bien ce travail.

Nous adressons nos remerciements les plus sincères à :

- **Mr ILBOUDO A**, notre encadrant, pour ses conseils précieux et sa patience.
- **La direction de l'ESTA**, pour avoir mis à notre disposition le cadre et les ressources nécessaires.

Le personnel de la bibliothèque, pour avoir partagé avec nous les réalités du terrain et les contraintes du métier.

RESUME

Ce rapport présente la démarche complète de conception et de réalisation d'une application Web et Desktop pour la gestion de la bibliothèque de l'ESTA. Face aux limites de la gestion manuelle actuelle, nous avons proposé une solution numérique robuste basée sur une architecture Spring Boot (Java) et React. Notre analyse s'appuie sur une modélisation UML exhaustive composée de 22 diagrammes, garantissant une couverture totale des processus métiers : de la sécurisation des accès (2FA) à la gestion fine des cycles de vie des emprunts et réservations.

LISTE DES SIGLES ET ABREVIATIONS

2FA : Two-Factor Authentication (Authentification à deux facteurs).

API : Application Programming Interface .

ESTA : École Supérieure des Techniques Avancées .

JPA : Java Persistence API.

JWT : Json Web Token.

REST : Representational State Transfer .

SIGB : Système Intégré de Gestion de Bibliothèque.

TOTP : Time-based One-Time Password.

UML : Unified Modeling Language.

ORM : Object-Relational Mapping.

CRUD : Create, Read, Update, Delete.

UI : User Interface (Interface utilisateur).

UX : User Expérience (Expérience utilisateur).

HTTP : HyperText Transfer Protocol.

HTTPS : HyperText Transfer Protocol Secure.

SQL : Structured Query Language.

JSON : JavaScript Object Notation.

DTO : Data Transfer Object.

DAO : Data Access Object.

Frontend : Partie visible de l'application.

Backend : Partie logique et serveur.

Cron Job : Tâche planifiée automatique.

LISTE DES FIGURES

Figure 1: Diagramme de cas d'utilisation	9
Figure 2: Inscription + vérification.....	10
Figure 3 : Connexion (Login + 2FA).....	11
Figure 4 : Setup & Activation 2FA	12
Figure 5 : Refresh Token.....	13
Figure 6 : Cycle de vie d'un emprunt.....	14
Figure 7 : Renouvellement d' emprunt.....	15
Figure 8 : Réservation de livre.....	16
Figure 9 : Annulation de réservation.....	17
Figure 10 : Scheduler Retards & Rappels (8h00).....	18
Figure 11 : Compte utilisateur	18
Figure 12 : Emprunt.....	19
Figure 13 : Réservation.....	20
Figure 14 : Cotisation	21
Figure 15 : Notifications	22
Figure 16 : Vue d'ensemble.....	23
Figure 17 : Réservation vers emprunt.....	24
Figure 18 : Diagramme de Classes	25
Figure 19 : Diagramme d'objet	26
Figure 20 : Architecture logicielle	27
Figure 21 : Composant de backend	28
Figure 22 : Navigation frontend	28

INTRODUCTION GENERALE

Dans un monde de plus en plus numérisé, la gestion manuelle des flux d'information devient un frein majeur à l'efficacité organisationnelle. Les bibliothèques universitaires, centres névralgiques du savoir, n'échappent pas à cette règle. À l'ESTA, la gestion des collections et des prêts nécessite une rigueur constante pour éviter la perte d'ouvrages et assurer une rotation équitable entre les étudiants.

La problématique centrale de ce projet est donc : Comment concevoir un système automatisé capable de sécuriser les accès, de notifier les utilisateurs en temps réel et de fournir des outils de pilotage précis aux bibliothécaires ?

Pour répondre à cette question, nous avons structuré notre travail en quatre chapitres :

- Le cadre général et l'étude de l'existant.
- L'analyse détaillée et la modélisation UML
- La conception technique et l'architecture logicielle.
- La présentation de la solution et le bilan.

CHAPITRE I : STRUCTURE D'ACCUEIL

I. Présentation de l'institution

L'ESTA est un établissement d'excellence formant les futurs cadres techniques du pays. La bibliothèque joue un rôle central dans l'accompagnement des étudiants

II. Justificatif du choix de la structure d'accueil

Analyse de processus actuel

Aujourd'hui, l'inscription se fait sur fiche cartonnée. L'emprunt consiste en une écriture manuelle dans un registre. Le suivi des retards dépend de la mémoire du bibliothécaire ou d'un feuilletage hebdomadaire du registre.

Limites identifiées

- **Incohérence** : Risque de prêter un livre déjà réservé.
- **Sécurité** : Aucune protection des données personnelles des étudiants.
- **Temps** : Processus long et fastidieux pour le personnel.

III. Objectifs du futur système

Nous visons la mise en place d'un SIGB moderne permettant la gestion automatisée des emprunts, le calcul des amendes, et une sécurité renforcée par authentification à deux facteurs.

CHAPITRE II : ANALYSE ET SPÉCIFICATIONS UML

C'est ici que nous détaillons la modélisation de notre système à travers nos 22 diagrammes produits.

I. Analyse fonctionnelle

L'analyse fonctionnelle constitue une étape centrale dans la conception du système. Elle permet d'identifier les besoins réels des utilisateurs, les processus métiers et les interactions entre les différents modules de l'application.

Le projet **Power Esta** vise à mettre en place un **Système Intégré de Gestion de Bibliothèque (SIGB)** moderne, sécurisé et automatisé.

Le système doit répondre aux contraintes suivantes :

- automatiser la gestion des livres ;
- sécuriser les accès ;
- optimiser les emprunts et réservations ;
- réduire les erreurs humaines ;
- améliorer le suivi administratif ;
- fournir des statistiques fiables.

Identification des acteurs

Le système implique plusieurs acteurs. Chaque acteur possède des responsabilités précises

Acteur	Description	Responsabilités
Visiteur	Utilisateur non authentifié	Consultation publique du catalogue
Étudiant	Lecteur inscrit	Emprunt, réservation, historique
Enseignant	Lecteur privilégié	Emprunt étendu, réservation prioritaire
Bibliothécaire	Gestionnaire opérationnel	Validation des prêts, retours, inventaire
Administrateur	Responsable système	Gestion utilisateurs, sécurité, stats
Système	Processus automatisé	Notifications, rappels, vérifications

Besoin fonctionnels

➤ Gestion des utilisateurs et sécurité

Le système doit permettre :

- inscription des utilisateurs ;
- la vérification des comptes par email ;
- authentification sécurisée ;
- la gestion des rôles et permissions ;
- l'activation de l'authentification à deux facteurs (2FA) par TOTP (application d'authentification) ou SMS ;
- le verrouillage automatique après plusieurs échecs ;
- le renouvellement sécurisé des sessions JWT.

Contraintes métier

- Un email doit être unique.
- Un compte non vérifié reste inactif.
- Après 5 tentatives échouées, le compte est verrouillé temporairement (déverrouillage par l'administrateur ou après un délai).

➤ Gestion du catalogue de livres

Le système doit assurer :

- ajout de livres ;
- la modification du catalogue ;
- la suppression des Ouvrages ;
- la catégorisation des livres ;
- la recherche multicritère ;
- l'affichage des disponibilités.

Critères de recherche : titre, auteur, ISBN, catégorie, langue.

Contraintes métier

- Un ISBN doit être unique.
- Un livre peut posséder plusieurs exemplaires.

➤ **Gestion des exemplaires**

Chaque livre physique est représenté par un exemplaire. Le système doit gérer :

- le status des exemplaires ;
- les codes-barres ;
- l'état physique des Ouvrages ;
- les acquisitions.

États possibles : disponible, emprunté, perdu, abîmé.

➤ **Gestions des emprunts**

Ce module représente le cœur du système. Le système doit permettre :

- la création d'emprunts ;
- la validation des prêts (par le bibliothécaire) ;
- le renouvellement (sous conditions) ;
- le retour des Ouvrages ;
- le calcul automatique des retards et des pénalités.

Vérifications métier avant validation :

- vérifier les quotas (maximum 3 emprunts simultanés pour un étudiant, 5 pour un enseignant) ;
- vérifier les retards existants ;
- vérifier la validité de l'abonnement ;
- vérifier la disponibilité d'au moins un exemplaire.

Contraintes

- Durée maximale d'emprunt configurable (ex. 14 jours pour un étudiant, 21 jours pour un enseignant).
- Nombre de renouvellements limité (2 maximum).
- Pénalités de retard : une amende forfaitaire fixe (ex. 1 000 FCFA par jour de retard) est appliquée automatiquement. L'utilisateur ne peut plus emprunter tant que l'amende n'est pas réglée.

➤ **Gestion des réservations**

Lorsqu'un livre n'est plus disponible, le système doit permettre une réservation.

Fonctionnalités :

- gestion des files d'attente ;
- calcul automatique des positions ;
- expiration automatique après 48h ;
- transformation réservation vers emprunt.

Contraintes

- Une réservation expire après 48h à compter de la notification de disponibilité. Si l'utilisateur ne retire pas l'ouvrage dans ce délai, la réservation est ****automatiquement annulée**** et l'exemplaire redevient disponible ; l'utilisateur peut être temporairement suspendu de réservation (pénalité de non-retrait).
- Les priorités (enseignant > étudiant) sont respectées dans la file d'attente.

➤ Gestion des notifications

Le système doit informer les utilisateurs automatiquement.

- **Types de notifications** : rappel de retard, disponibilité de réservation, confirmation d'emprunt, alerte d'expiration d'abonnement, notification de pénalité.
- **Canaux** : email, SMS, notifications push.
- **Automatisation** : un Scheduler quotidien déclenche les rappels, alertes et vérifications (retards, expirations, relances).

➤ Gestion des abonnements et cotisations

Le système doit gérer :

- les abonnements annuels selon le type d'utilisateur :
- Étudiant : tarif réduit, valable 1 an.
- Enseignant : gratuité ou tarif préférentiel, durée alignée sur le contrat académique.
- Externe : tarif plein, renouvellement manuel.
- le paiement en ligne (carte bancaire, mobile money) ;
- les renouvellements automatiques (avec accord préalable) ;
- les restrictions liées aux cotisations impayées.

Contrainte : un utilisateur dont l'abonnement n'est pas à jour ne peut pas emprunter

Le système doit offrir :

- Avis sur les livres : notation (1 à 5 étoiles), commentaires modérés, historique.
- Wishlist : sauvegarde personnelle, favoris.
- Listes de lecture : organisation personnalisée (thèmes, objectifs).
- Alertes thématiques : notifications selon les préférences de l'utilisateur.

➤ Gestion des statistiques

Le système doit produire des indicateurs fiables pour les administrateurs :

- livres les plus empruntés ;
- utilisateurs les plus actifs ;
- taux de retard et montant total des amendes ;
- répartition par catégorie ;
- historique des activités.

Objectif : aide à la décision (acquisitions, politique tarifaire, animation).

➤ Gestion des acquisitions

Le système doit gérer :

- les demandes d'acquisition (suggestions des utilisateurs ou du personnel) ;
- les fournisseurs ;
- les commandes ;
- le suivi des achats.

Workflow : demande vers validation (par le bibliothécaire ou l'administrateur) vers commande vers réception vers ajout au catalogue (création des exemplaires).

Besoins non fonctionnels

➤ Sécurité

- Authentification JWT + Spring Security.
- Authentification à deux facteurs (2FA) optionnelle.
- Contrôle fin des permissions (RBAC).

➤ Performance

- Temps de réponse < 500 ms pour les opérations courantes.
- Pagination systématique des listes.
- Optimisation des requêtes (index, caches).

➤ Disponibilité

- Accès 24h/24, 7j/7.
- Sauvegardes automatisées quotidiennes de la base de données.

➤ **Maintenabilité**

- Architecture modulaire (backend Spring Boot, frontend React).
- Séparation claire entre couches (MVC).
- Documentation API (Swagger/OpenAPI).

➤ **Portabilité**

- Application web responsive.
- Application desktop via Tauri (hors ligne avec synchronisation).
- Compatible Linux, Windows, macOS.

Synthèse fonctionnelle

Le système **Power Esta** constitue une plateforme complète de gestion de bibliothèque. Il intègre sécurité avancée, automatisation métier, gestion documentaire, interactions utilisateurs et outils décisionnels. L'analyse fonctionnelle démontre qu'il ne s'agit pas d'une simple gestion de livres, mais bien d'un système d'information intelligent, sécurisé et évolutif.

Diagramme de cas d'utilisation

Ce diagramme identifie les fonctions clés : consultation du catalogue (Public), réservation et suivi (Étudiant/Enseignant), validation des prêts et inventaire (Bibliothécaire), et paramétrage (Admin).

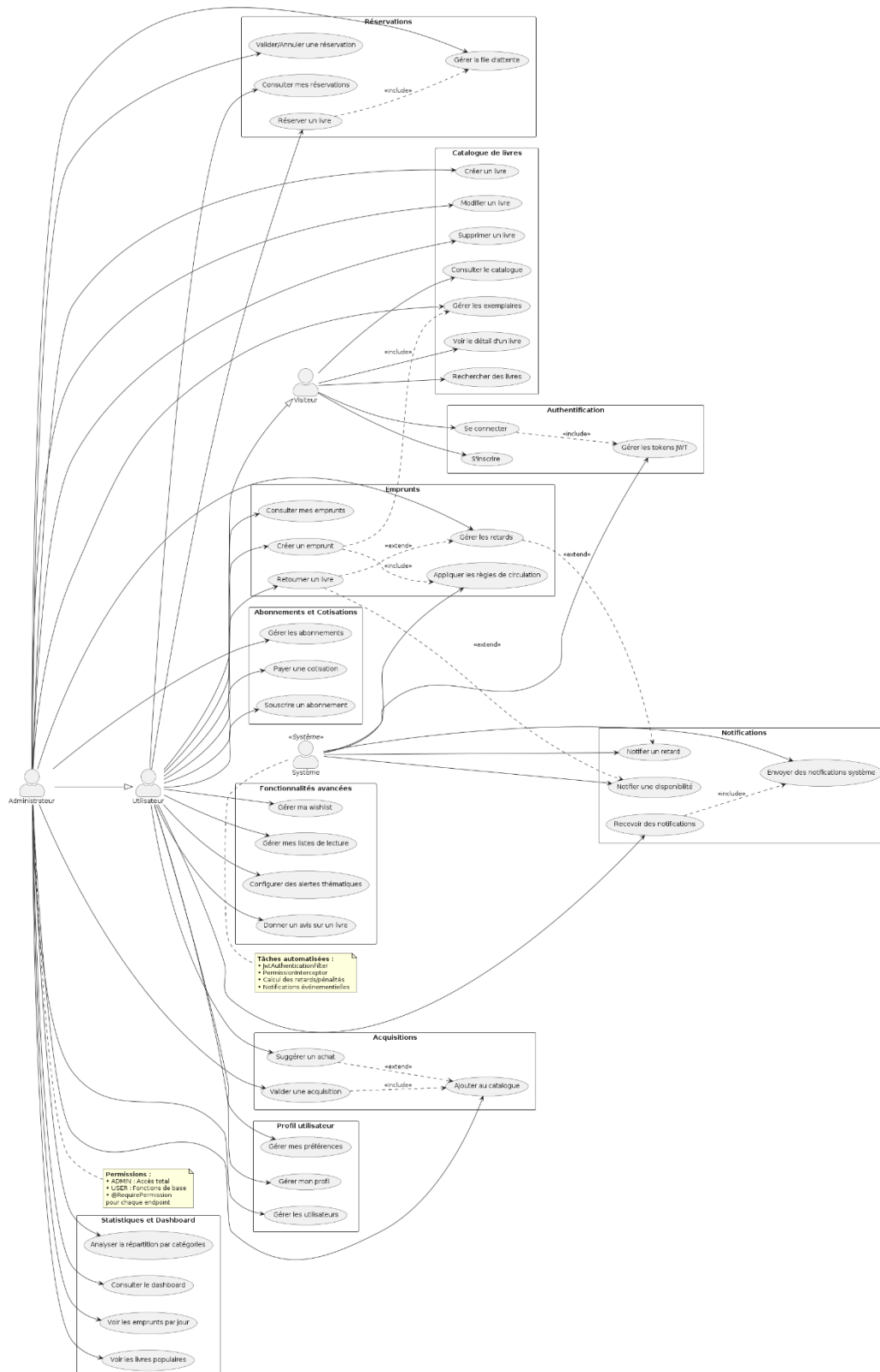


Figure 1: Diagramme de cas d'utilisation

II. Modélisation dynamique

Inscription + Vérification

Ce flux montre la création d'un compte utilisateur en état "Inactif". Un service génère un Token cryptographique envoyé par mail. Le compte n'est activé que lorsque l'utilisateur clique sur le lien, garantissant la validité des coordonnées de contact pour les futurs rappels.

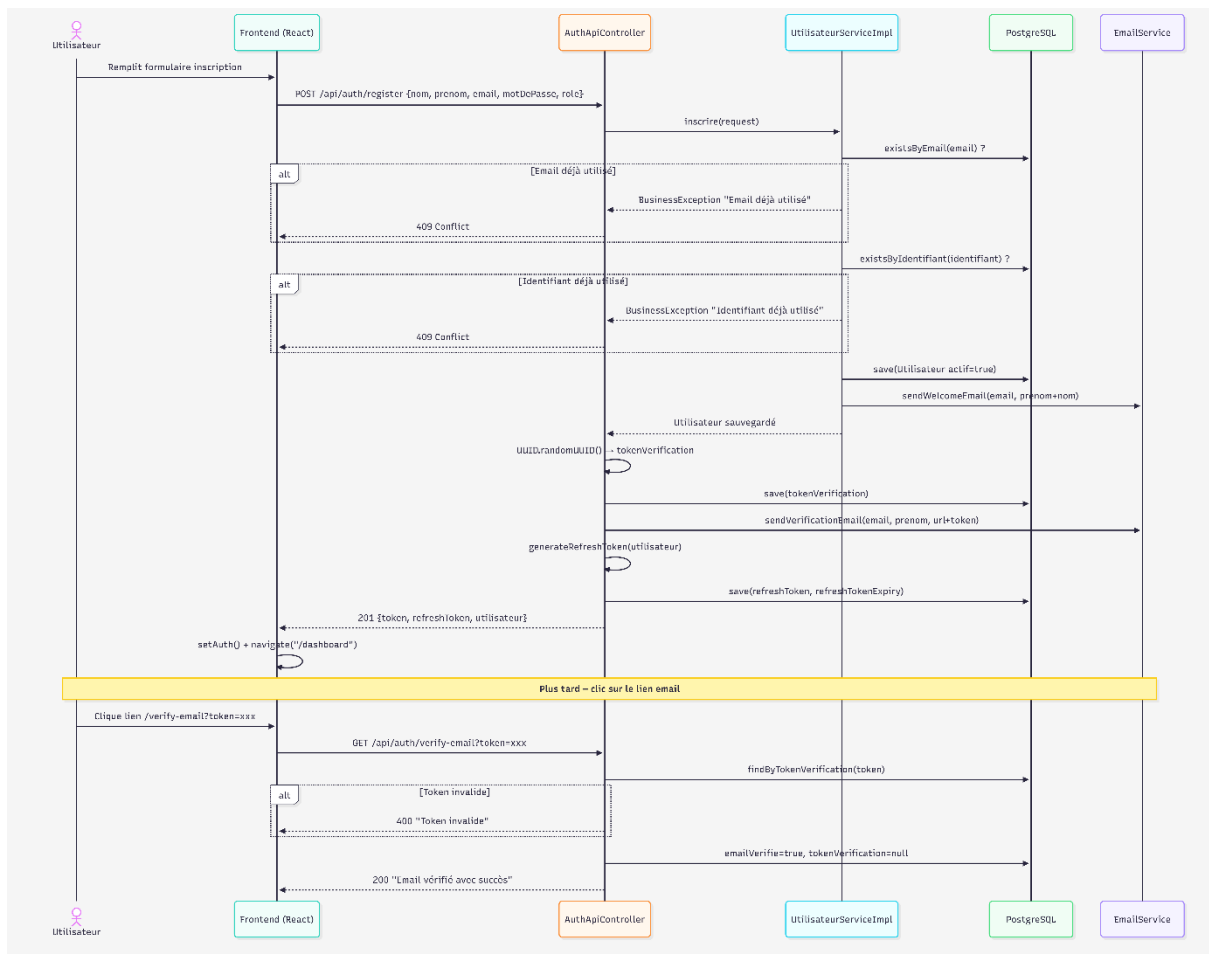


Figure 2: Inscription + vérification

Connexion (Login + 2FA)

Nous avons implémenté une sécurité de niveau bancaire. Après validation du mot de passe (Bcrypt), le système vérifie si le 2FA est actif. Si oui, un challenge TOTP est envoyé. L'accès n'est libéré qu'après validation du code temporaire.

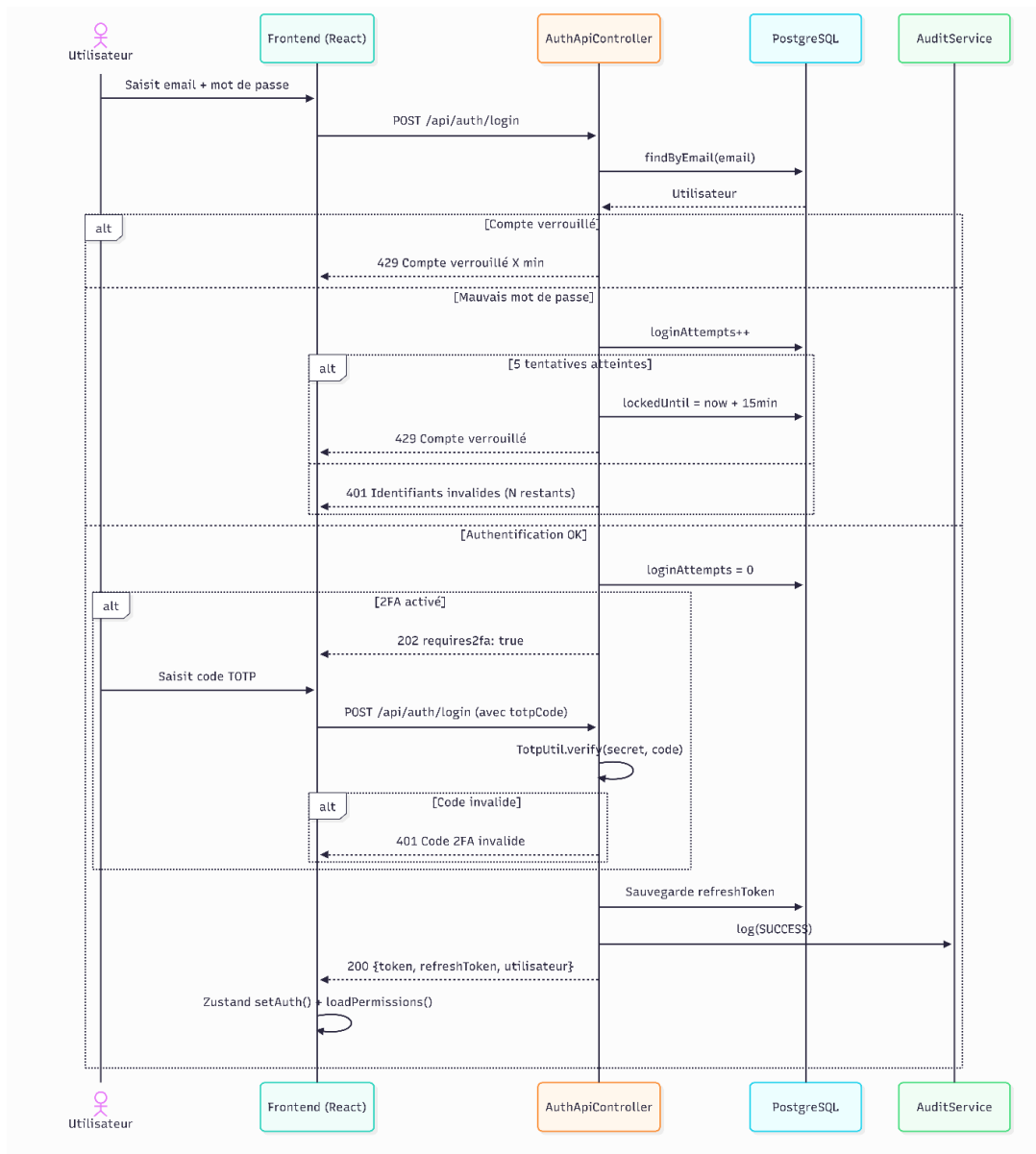


Figure 3 : Connexion (Login + 2FA)

Setup & Activation 2FA

Détaille le processus d' enrôlement. Le backend génère une clé secrète partagée, affichée sous forme de QR Code via le frontend. L'utilisateur scanne ce code avec une application comme Google Authenticator.

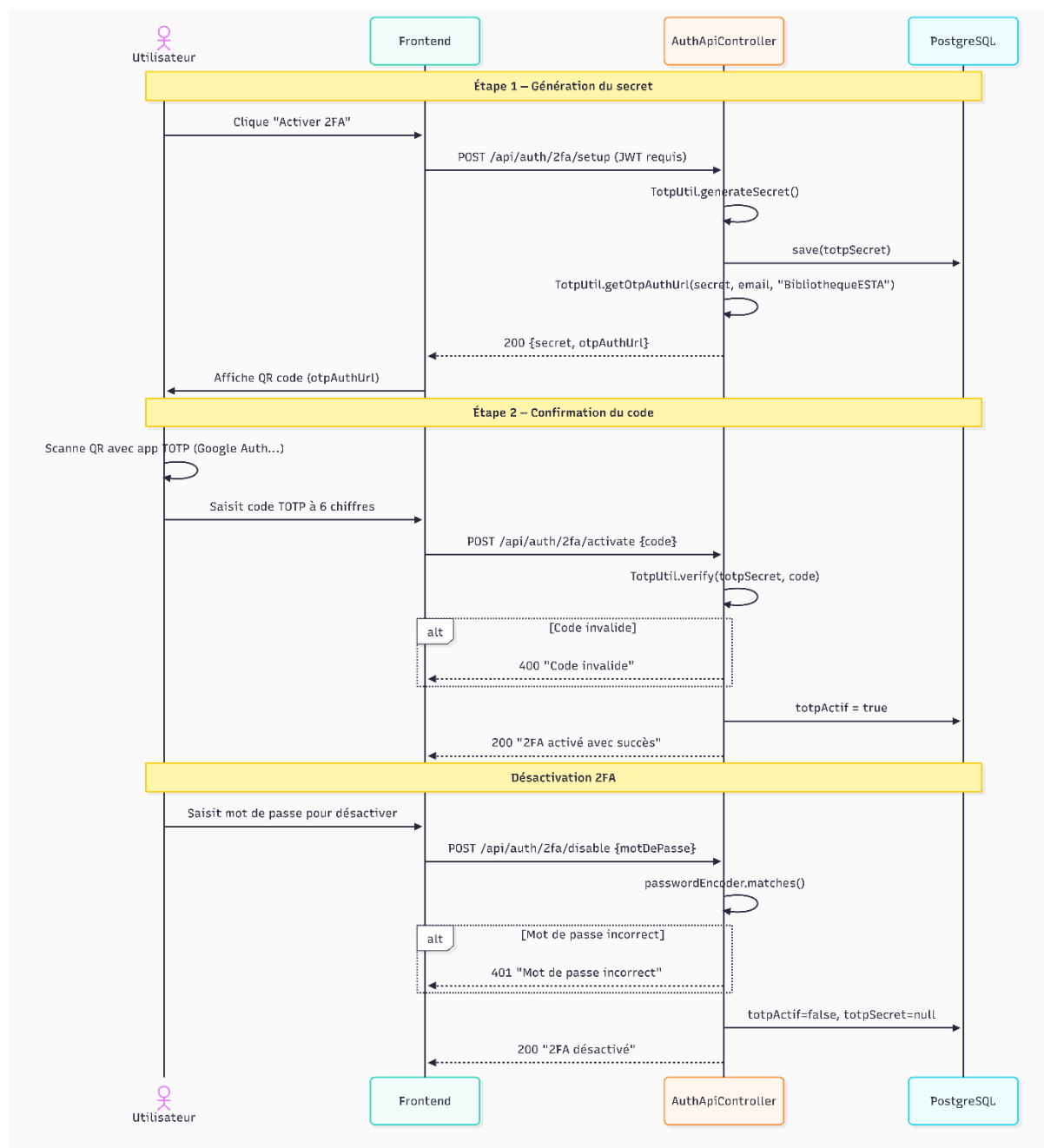


Figure 4 : Setup & Activation 2FA

Refresh Token

Pour améliorer l'expérience utilisateur, nous utilisons des "Refresh Tokens" persistés en base de données. Cela permet de renouveler l'Access Token (JWT) sans forcer l'utilisateur à ressaisir son code 2FA trop souvent.

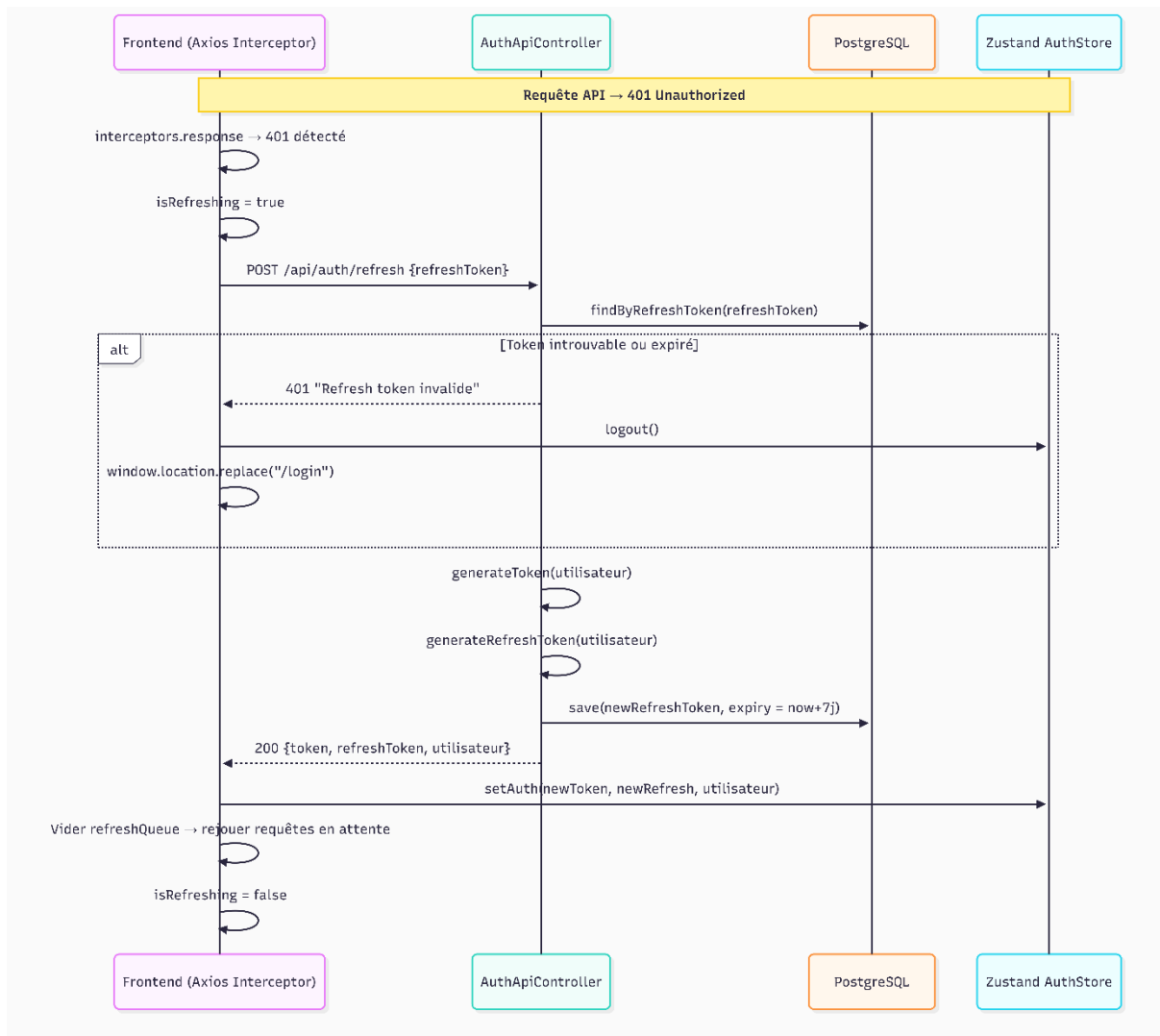


Figure 5 : Refresh Token

Cycle de vie d'un emprunt

C'est le cœur métier. Le diagramme montre les vérifications de quota (max 3 livres) et d'absence de retards avant de valider la sortie. L'exemplaire est marqué comme Indisponible instantanément.



Figure 6 : Cycle de vie d'un emprunt

Renouvellement d'emprunt

Un utilisateur peut prolonger son prêt via son interface. Le système vérifie deux conditions : le livre ne doit pas être réservé par un tiers et le nombre maximum de renouvellements ne doit pas être atteint.

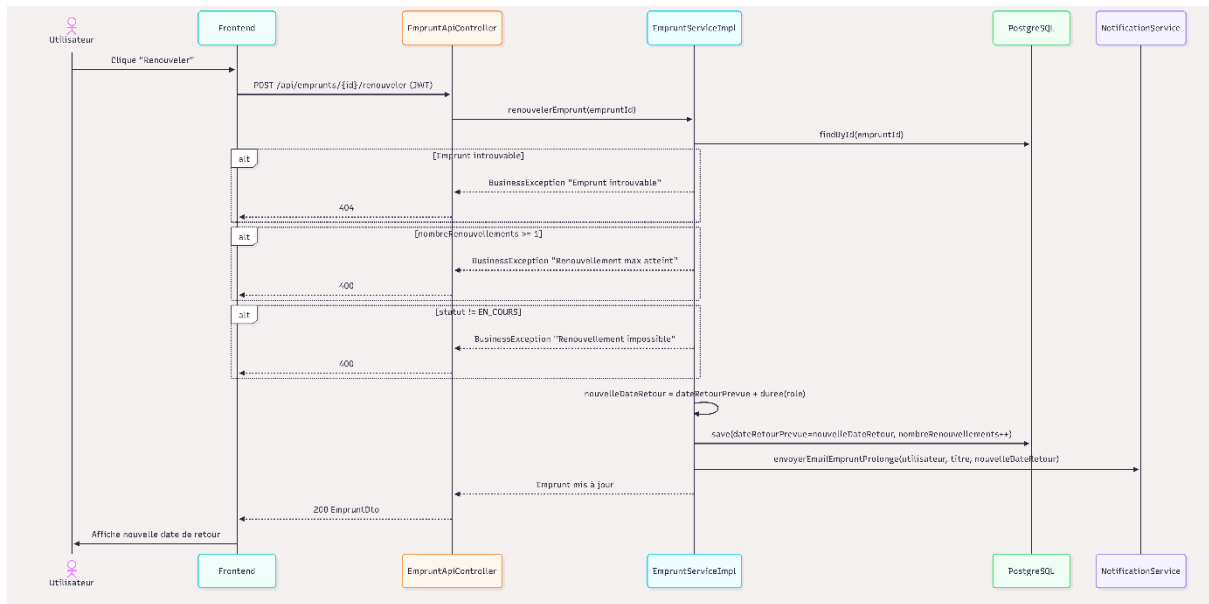


Figure 7 : Renouvellement d'emprunt

Réservation de livre

Si tous les exemplaires d'un titre sont empruntés, le bouton Réserver apparaît. Le système calcule la position de l'utilisateur dans la file d'attente et lui envoie une confirmation par email.

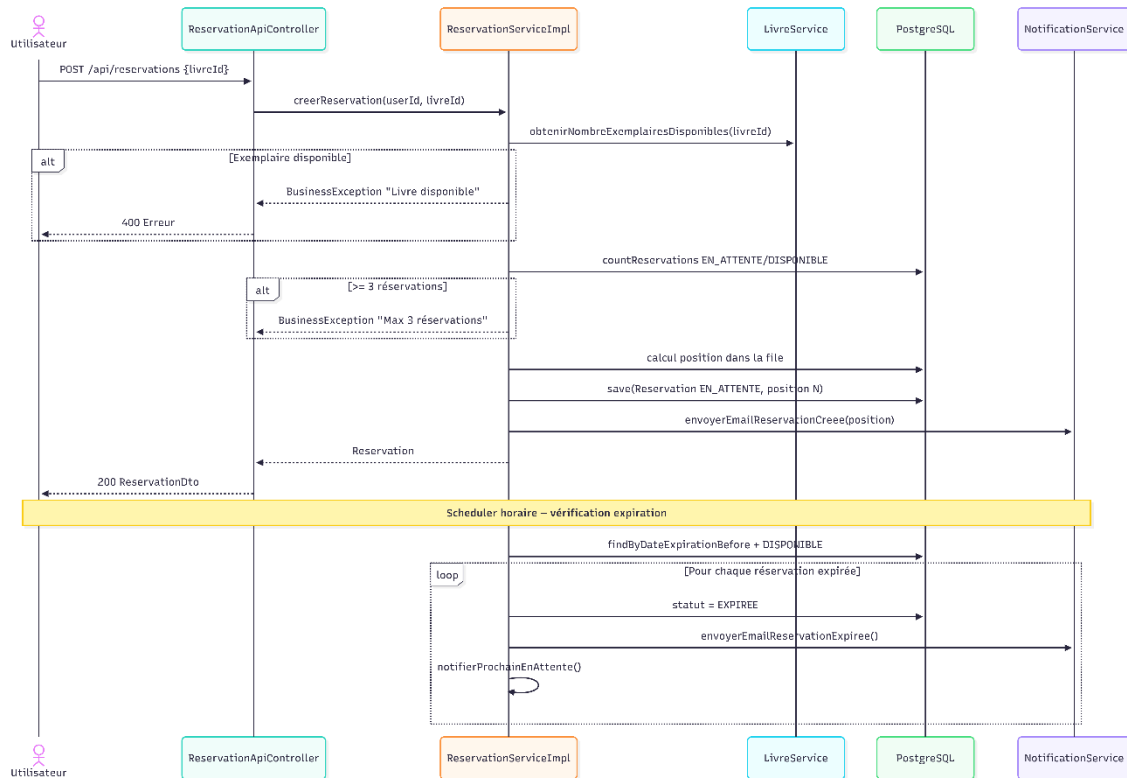


Figure 8 : Réserve de livre

Annulation de réservation

Cette fonctionnalité permet de gérer les désistements. Le système réorganise automatiquement la file d'attente et décale les positions des autres réservataires.

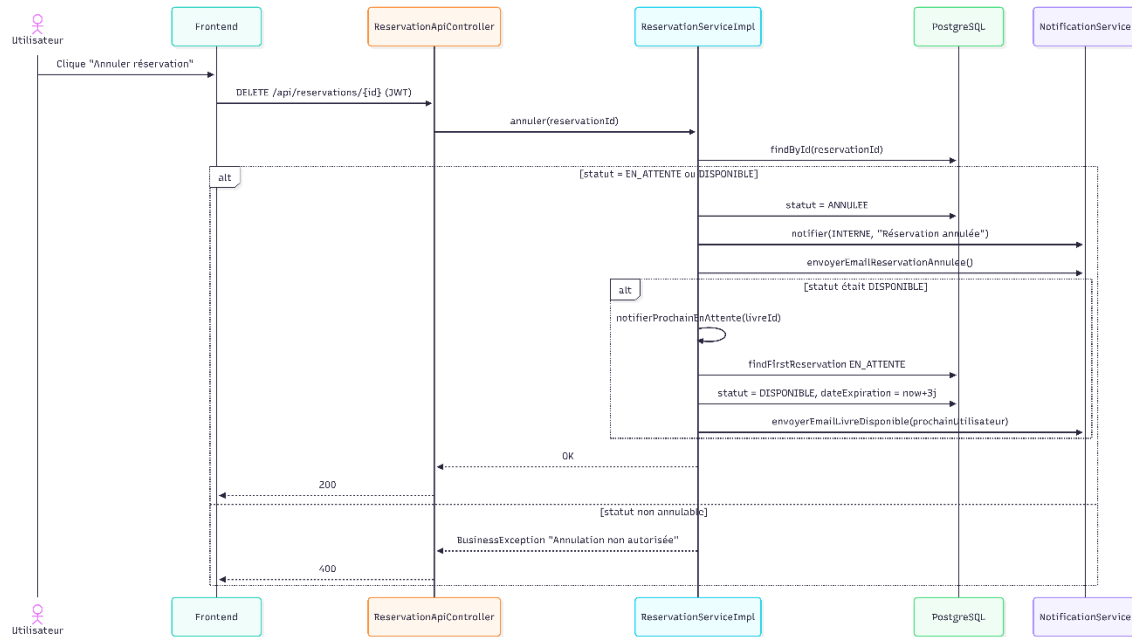


Figure 9 : Annulation de réservation

Scheduler Retards & Rappels (8h00)

Une tâche automatisée (Cron Job) scanne la base chaque matin à 8h00. Elle identifie les emprunts dont la date de retour est dépassée et déclenche l'envoi massif de notifications de rappel.

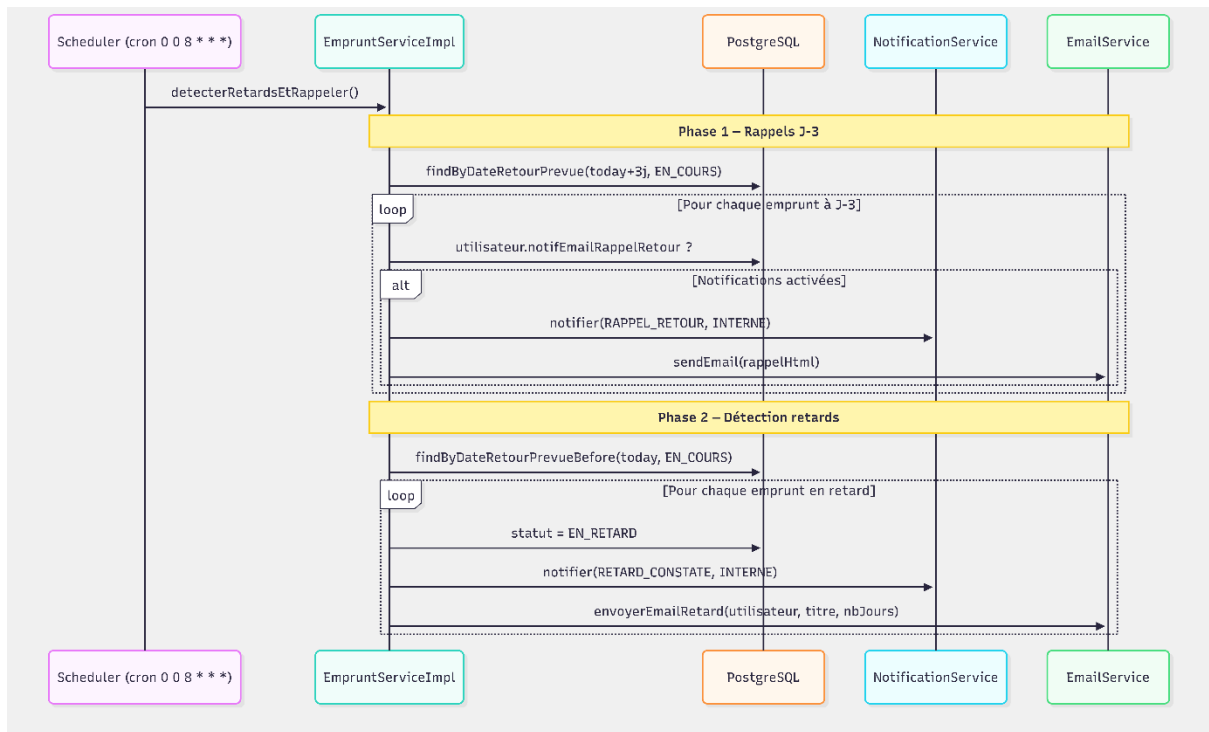


Figure 10 : Scheduler Retards & Rappels (8h00)

III. Modélisation des états (Les 5 cycles de vie)

Compte utilisateur

Passage de l'état "EN_ATTENTE_VERIF" à "ACTIF", avec possibilité de passage en "VERROUILLE" après 5 échecs de connexion.

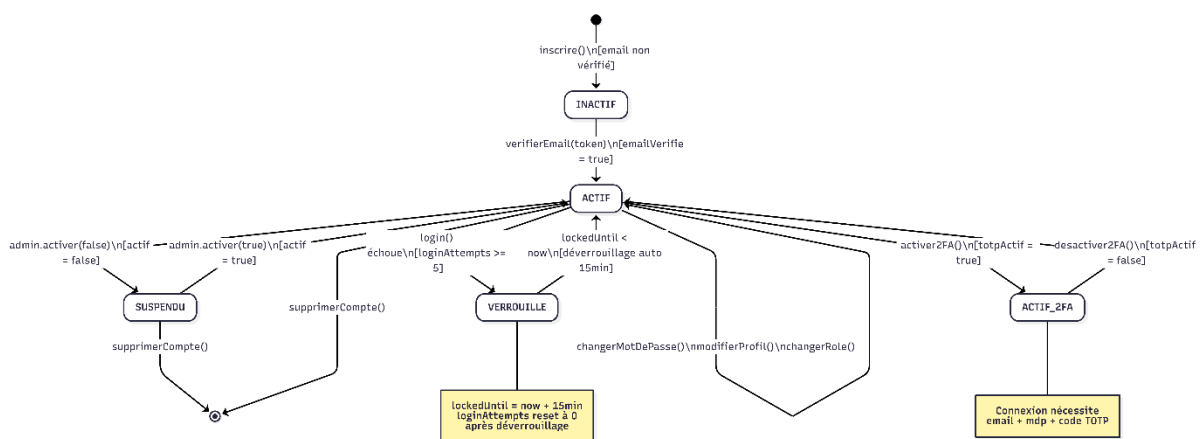


Figure 11 : Compte utilisateur

Emprunt

Un emprunt naît en état "EN_COURS". Il peut devenir "EN_RETARD", puis se terminer en "RETOURNE" ou "PERDU".

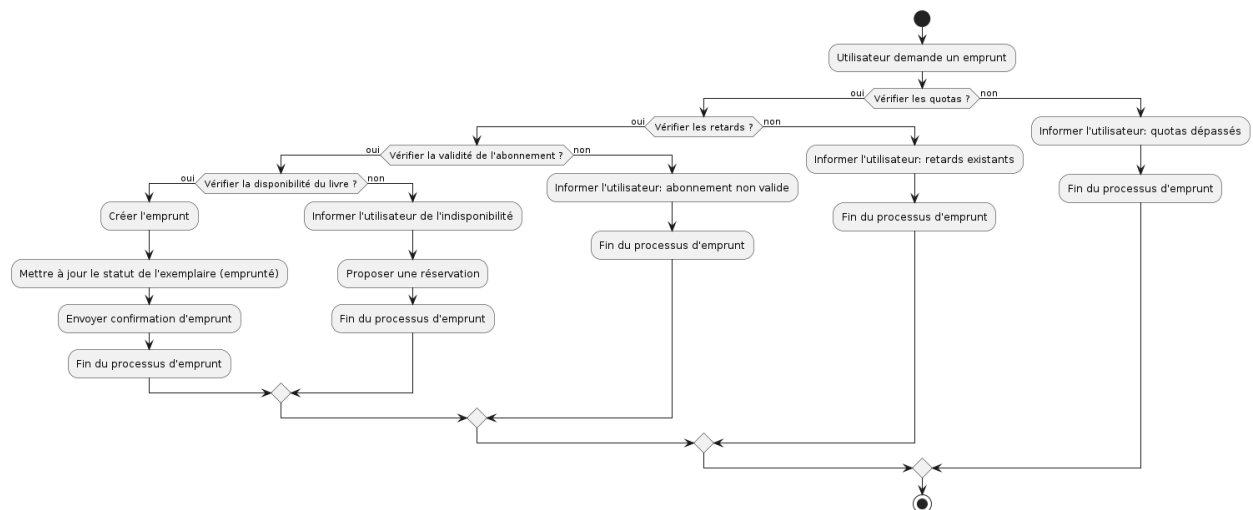


Figure 12 : Emprunt

Réservation

Cycle allant de "EN_ATTENTE" à "DISPONIBLE" (quand le livre est rendu), puis "CONFIRMEE" ou "EXPIREE" (si l'utilisateur ne vient pas chercher le livre sous 48h).

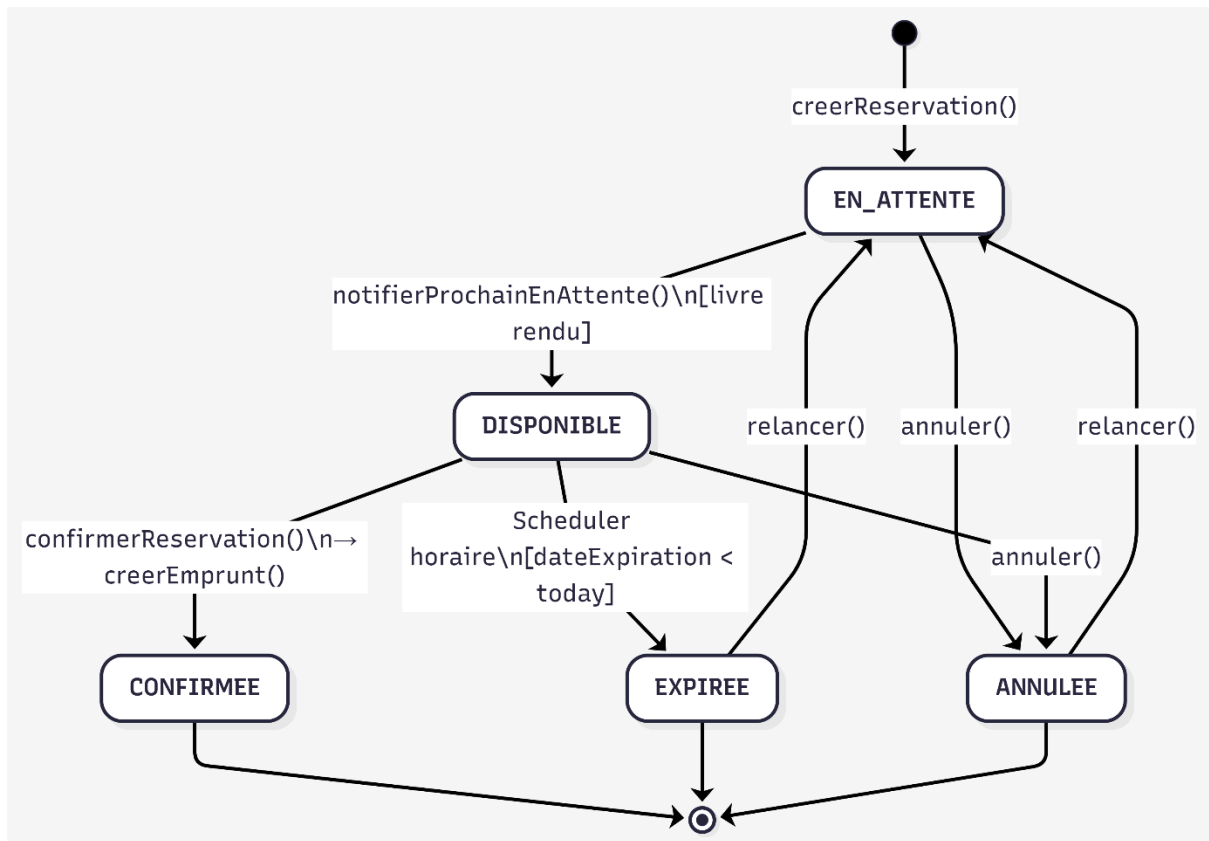


Figure 13 : Réserveation

Cotisation

C'est l'état de l'abonnement annuel de l'étudiant. Il conditionne le droit d'emprunt.

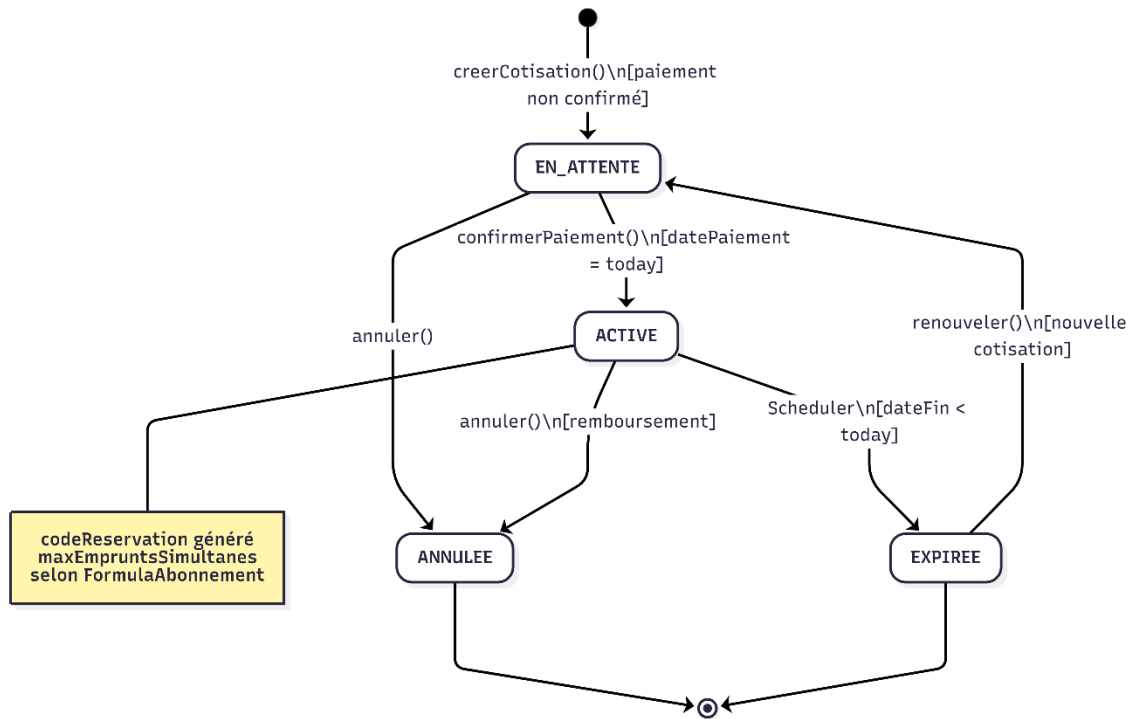


Figure 14 : Cotisation

Notifications

L'utilisateur suit le message de sa création à son marquage comme "Lu" par l'utilisateur final

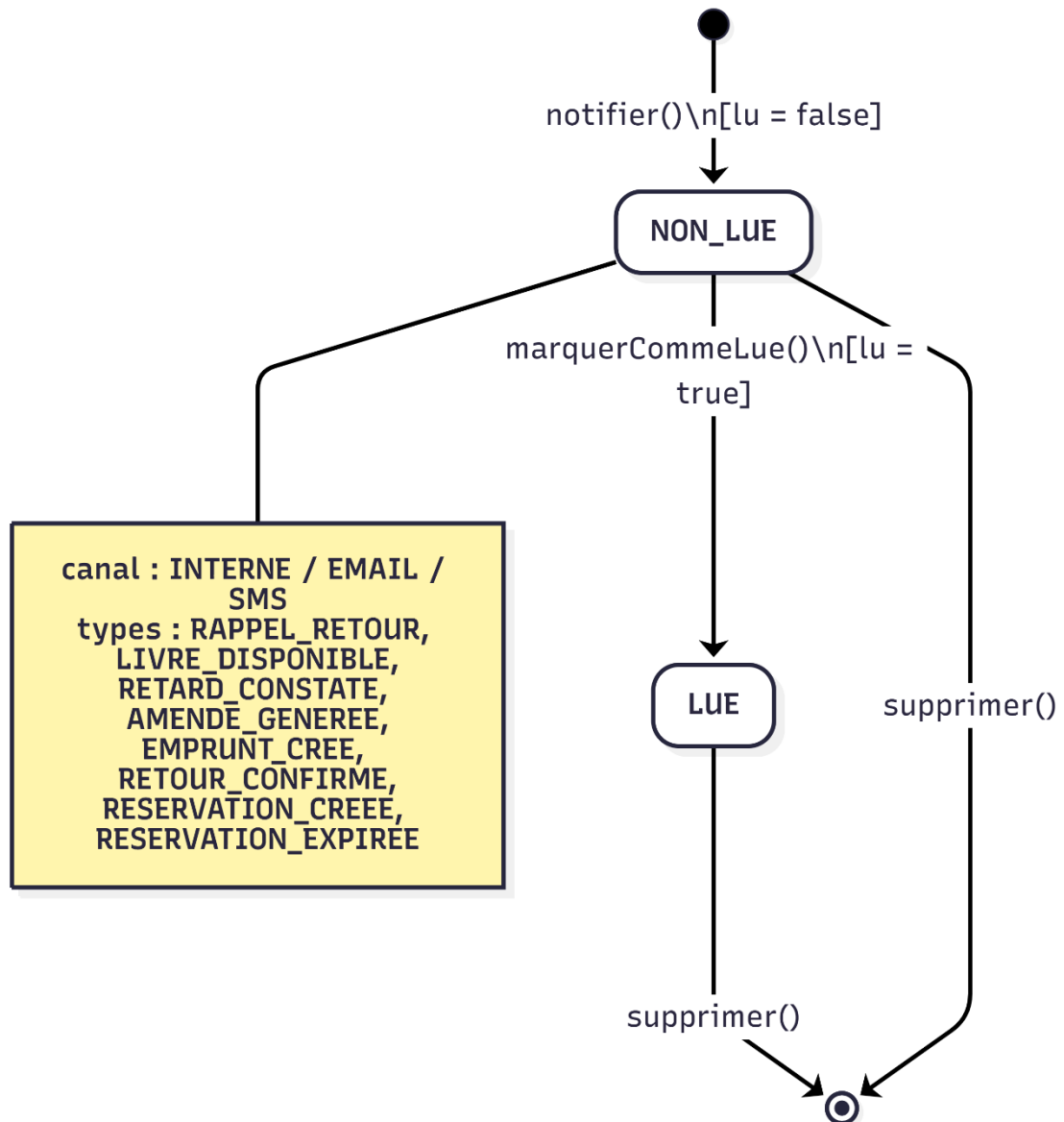


Figure 15 : Notifications

IV. Interactions et Structure Statique

- **Vue d'ensemble :** Il montre comment le module Utilisateur communique avec le module Emprunt via des interfaces de service.

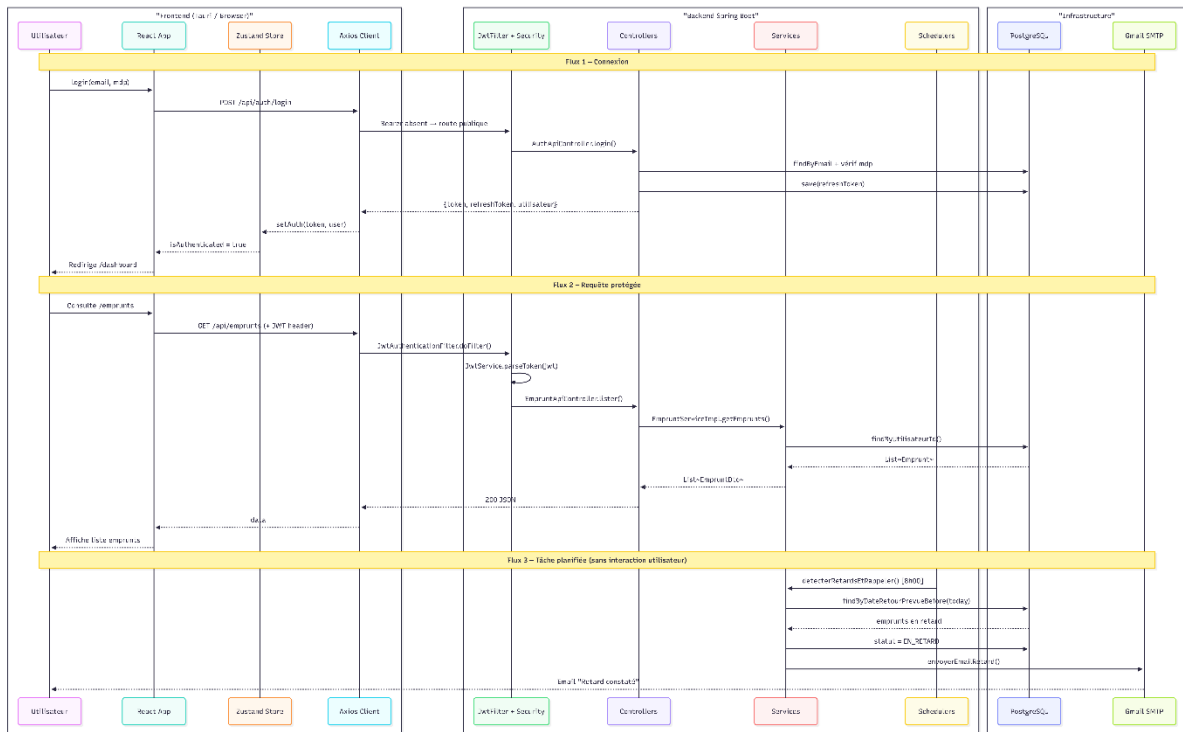


Figure 16 : Vue d'ensemble

- **Réservation vers Emprunt :** Il détaille le mécanisme de transfert de données lorsqu'une réservation prioritaire se transforme en prêt effectif.

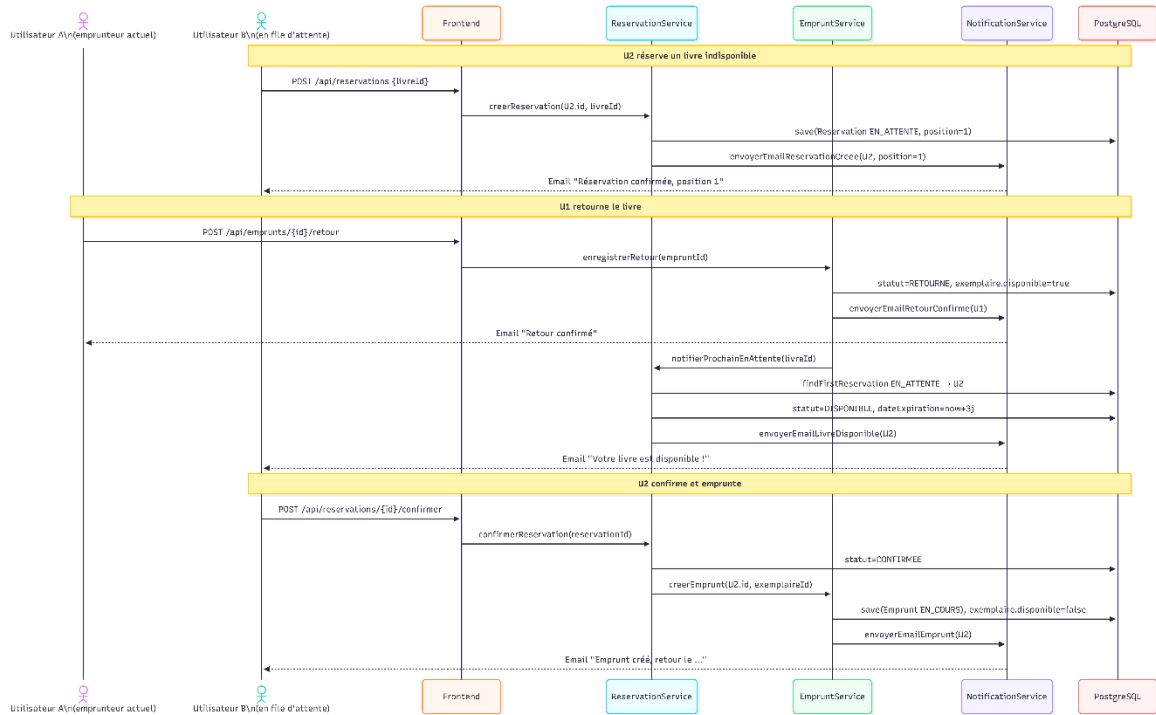


Figure 17 : Réserve vers emprunt

- **Diagramme de Classes :** Il présente les 15 entités majeures (Livre, Utilisateur, Role, Exemplaire, etc.) et leurs multiplicités.

Figure 19 : Diagramme d'objet

CHAPITRE III : CONCEPTION TECHNIQUE

I. Architecture logicielle

Nous avons opté pour une architecture web moderne.

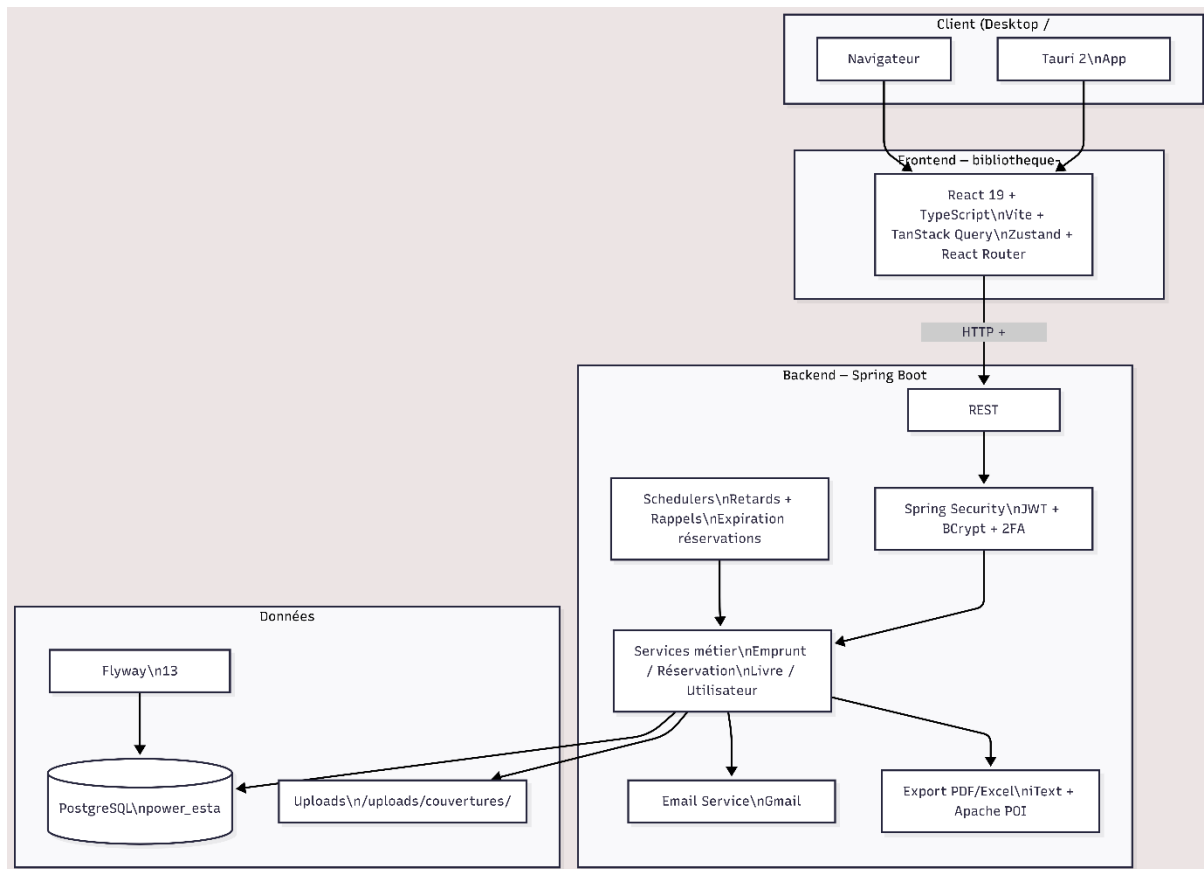


Figure 20 : Architecture logicielle

- **Backend** : Spring Boot 3.5.4, utilisant JPA pour l'ORM et Spring Security pour la protection.
- **Frontend** : React 19 pour une interface fluide, packagée avec Tauri 2 pour un rendu desktop natif.

II. Composants et Navigation

- **Composants Backend :** Ici nous avons une séparation stricte entre les API Controllers, les Services métier et les Data Repositories.

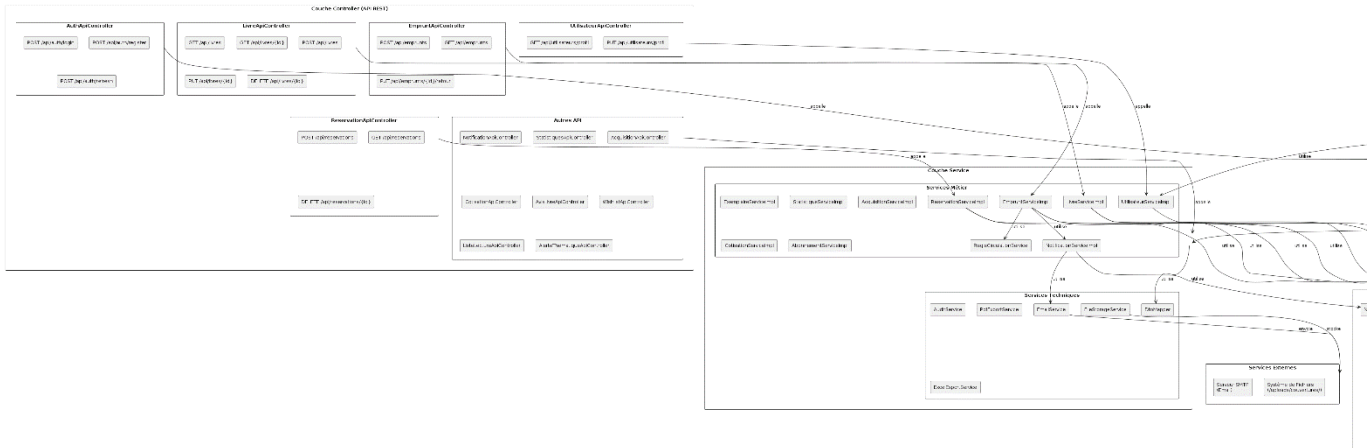


Figure 21 : Composant de backend

- **Navigation Frontend :** Le schéma des routes sécurisées interdisant l'accès aux pages d'administration aux simples lecteurs.

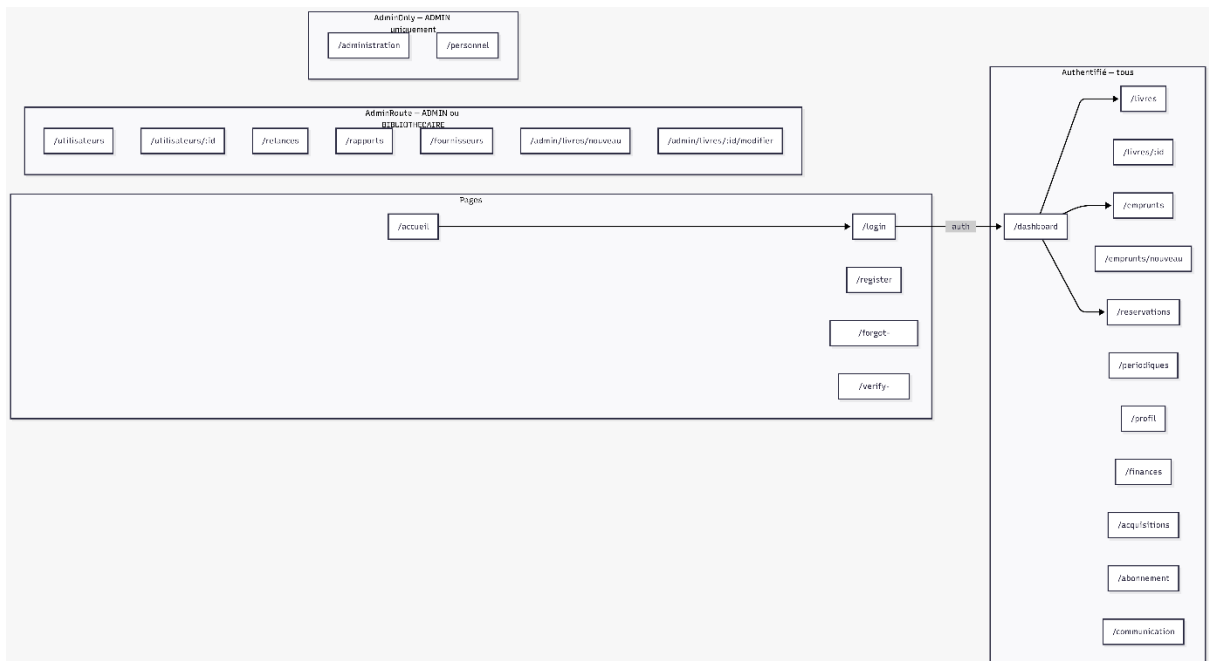


Figure 22 : Navigation frontend

III. Modèle de la Base de Données(Schéma Relationnel)

Pour soutenir les 15 entités identifiées dans le diagramme de classes, un modèle relationnel robuste a été conçu. Le schéma ci-dessous décrit les tables principales, leurs clés primaires (PK) et étrangère (FK), illustrant les relations entre les données.

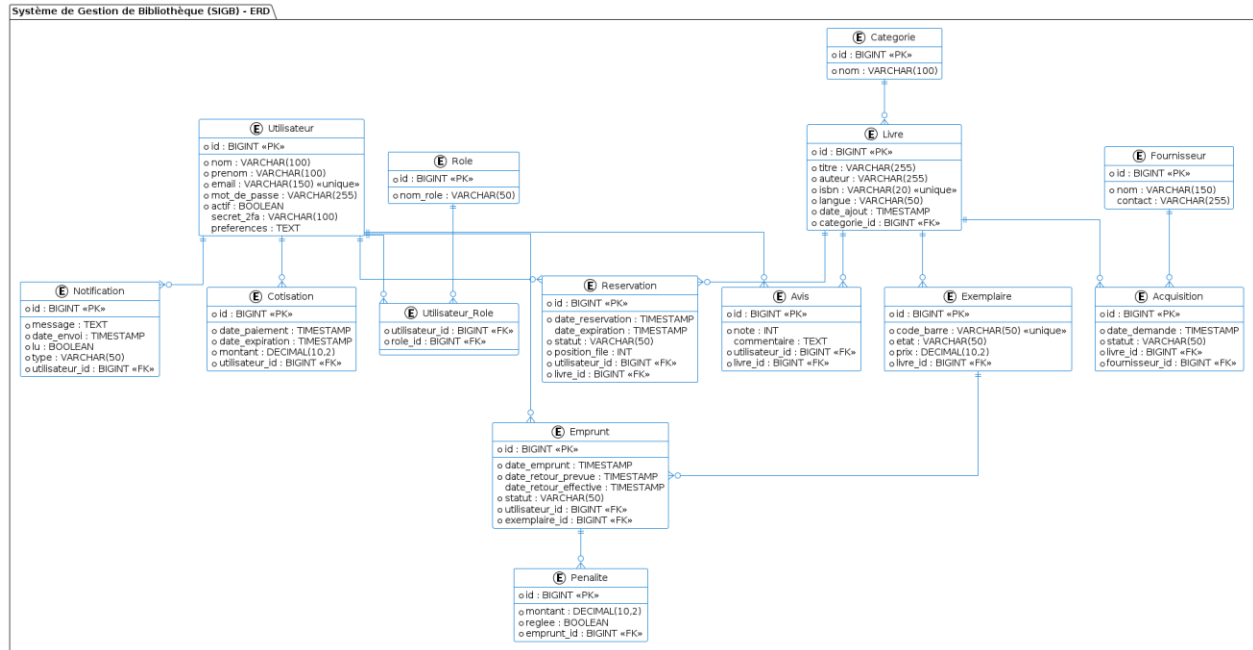


Figure 23: diagramme ERD

Description des relations clés :

- **Utilisateur à Emprunt (1 : N)** : Un utilisateur peut effectuer plusieurs emprunts, mais un emprunt est lié à un seul utilisateur. La table **Emprunt** contient **utilisateur_id** comme clé étrangère.
- **Livre à Exemple (1 : N)** : Un livre du catalogue peut posséder plusieurs exemplaires physiques. La table **Exemple** contient **livre_id** comme clé étrangère.
- **Exemple à Emprunt (1 : N)** : Un exemplaire physique peut être emprunté plusieurs fois au cours de sa vie (historique), mais un enregistrement d'emprunt spécifique concerne un seul exemplaire. La table **Emprunt** contient **exemple_id** comme clé étrangère.
- **Utilisateur à Rôle (N : M)** : Géré via une table jointure **Utilisateur_Role** pour permettre un utilisateur d'avoir plusieurs rôles (exemple dans notre cas , nous avons : Bibliothécaire, Enseignant , Etudiant , Public , Administrateur).

IV. Justification des Choix Technologiques

Le choix des technologies pour ce projet n'est pas fortuit ; il répond à des exigences strictes de performance, de sécurité et de maintenabilité.

- Spring Boot (Backend) : Nous avons choisi Spring Boot car il offre une injection de dépendances native, de sécurité et de maintenabilité. Son intégration fluide avec gestion des tâches planifiées (Cron Jobs) via l'annotation `@Scheduled` est idéale pour notre système de notifications automatiques (retard, expirations).
- React (Frontend Web) : React a été sélectionné pour sa capacité à créer des interfaces utilisateur dynamiques et réactives grâce à son DOM virtuel . Son écosystème riche et son approche par composant permettent une réutilisation du code et une maintenance aisée .
- Tauri(Frontend Desktop) : Contrairement à Electron , Tauri utilise les webviews natives du système d'exploitation ,ce qui permet de générer des exécutables beaucoup plus légers et performants, tout en réutilisant le code React du frontend web . Cela répondait parfaitement à notre besoin d'une application desktop rapide .
- PostgreSQL (Base de données) : Ce SGBDR a été privilégié pour sa robustesse, sa conformité aux standards SQL et sa capacité à gérer des transactions complexes de manière fiable (ACID), ce qui est crucial pour un système gérant des emprunts et des réservations concurrentes.
- JWT (JSON Web Token) : Utilisé pour l' authentification stateless . Il permet de sécuriser les API REST sans surcharger le serveur avec la gestion des sessions , facilitant ainsi la scalabilité de l'application .

V. Documentation des API REST

Le backend expose une série d'API RESTful pour la communication avec les clients (Web et Desktop). Voici un extrait des endpoints principaux :

Méthode	Endpoint	Rôle Requis	Description
POST	/api/auth/register	Public	Inscription d'un nouvel utilisateur.
POST	/api/auth/login	Public	Authentification et génération du Token JWT
POST	/api/auth/2fa/verify	Public (avec Token temporaire)	Validation du code TOTP pour le 2FA.
GET	/api/livres	Public / Etudiant	Recherche multicritère dans le catalogue (titre, auteur, etc.).
POST	/api/emprunts	Bibliothécaire	Création et validation d'un nouvel emprunt.
PUT	/api/emprunts/{id}/retour	Bibliothécaire	Enregistrement du retour d'un exemplaire.
POST	/api/reservations	Etudiant / Enseignant	Création d'une réservation pour un livre indisponible.
GET	/api/utilisateurs/{id}/historique	Etudiant (soi-même) / Admin	Consultation de l'historique des emprunts.

CHAPITRE IV : REALISATION ET IMPLEMENTATION

Ce chapitre reprend et enrichit la section de réalisation, en y ajoutant les éléments essentiels pour la présentation et l'évaluation du projet.

I. Environnement de Développement et Technologies Utilisées

Outil / Technologie	Version	Rôle dans le projet
Vs code	1.120.0	IDE Frontend
Spring Boot	3.5.4	Framework Backend
React + TypeScript	19 + Vite	Framework Frontend
Tauri	1.0.1	Packaging Desktop
PostgreSQL	18	Base de données
Flyway	13	Migration BDD
Postman	12.8.2	Test des API REST

Git / GitHub	2.52.0.windows.1	Versionement
Maven	3.9.13	Gestion de dépendances

II. Description des Modules Clés

- **Module Authentification** : Implémenté via AuthApiController, il gère l'inscription, la connexion, le 2FA TOTP, les Refresh Token et le verrouillage de compte après 5 tentatives échouées. La sécurité repose sur Spring Security + BCrypt pour le hachage des mots de passe et JWT pour les sessions stateless.
- **Module Catalogue de Livres** : Géré par LivreServiceImpl, il assure les opérations CRUD sur les livres et leurs exemplaires physiques. Il offre une recherche multicritère (titre, auteur, ISBN, catégorie, langue) et maintient en temps réel le statut de disponibilité de chaque exemplaire (disponible, emprunté, perdu, abîmé).
- **Module Emprunt** : Cœur du système, géré par EmpruntServiceImpl. Avant toute validation, il vérifie automatiquement les quotas (max 3 pour un étudiant, 5 pour un enseignant), l'absence de retards existants et la validité de l'abonnement. Il gère également le renouvellement (2 maximum) et le calcul automatique des pénalités de retard (1 000 FCFA/jour).
- **Module Réservation** : Implémenté via ReservationServiceImpl, il permet à un utilisateur de rejoindre une file d'attente lorsqu'un livre est indisponible. Il calcule automatiquement la position dans la file en respectant les priorités (enseignant > étudiant), et transforme la réservation en emprunt dès que l'exemplaire devient disponible. Une réservation non confirmée expire automatiquement après 48h.
- **Module Notifications** : Géré par NotificationService et EmailService, ce module gère l'envoi automatique de messages via email. Les types couverts incluent : confirmation d'emprunt, rappel de retard, disponibilité de réservation, expiration d'abonnement et notification de pénalité. Chaque notification suit un cycle de vie de l'état NON_LUE à LUE.

Champ	Valeur	Signification
Secondes	0	à la seconde 0
Minutes	0	à la minute 0
Heures	8	à 8h00
Jours du mois	*	tous les jours
Mois	*	tous les mois
Jour de la semaine	*	tous les jours de la semaine

- **Module Scheduler** : Tâche Cron (**tous les jours à 08h00:00 exactement**) exécutée chaque matin à 8h00 par `EmpruntServiceImpl`. Elle opère en deux phases : d'abord l'envoi de rappels J-3 avant la date de retour prévue, puis la détection des emprunts en retard avec mise à jour automatique du statut vers `EN_RETARD` et envoi des notifications correspondantes.
- **Module Périodiques** : Ce module gère les publications en série de la bibliothèque (revues, magazines, journaux scientifiques). Il repose sur trois entités principales :
`Periodique` : la notice de la publication avec son titre, ISSN, éditeur, langue, fréquence de parution et couverture.
`AbonnementPeriodique` : la relation contractuelle entre un périodique et un fournisseur, incluant les dates de validité, le montant et le statut de l'abonnement.
`EmpruntPeriodique` / `ReservationPeriodique` : la gestion des prêts et réservations de numéros spécifiques, distincts des livres classiques.

III. Présentation de la Solution pour les Tests

Pour faciliter l'évaluation et les tests de l'application par un tiers, notamment par le corps enseignant, il est crucial de fournir des instructions claires pour le déploiement et l'utilisation du système.

A. Prérequis Techniques

Avant de lancer l'application, assurez-vous que les environnements suivants sont installés et configurés :

- **Java Development Kit (Kit)** : Version 17.0.0.1
- **Node.js** : Version 24.12.0
- **PostgreSQL** : Une instance de base de données PostgreSQL doit être accessible. Les identifiants de connexion (URL, utilisateur, mot de passe) devront être configurés dans le fichier **application.properties** du backend.
- **Git** : Pour cloner le dépôt du projet.
- **Apache Maven** : Pour la gestion du projet backend.

B. Etapes de lancement

1. Cloner du dépôt : git clone https://github.com/Watta573/power_esta

2 Configuration de la Base de Données

2. Configuration de la Base de Données

A. Création de la base

Ouvrez votre terminal SQL (psql) ou un outil comme pgAdmin et créez la base de données :

```
CREATE DATABASE power_esta;
```



B. Importation des données

Le projet utilise **Flyway** pour gérer les migrations, mais vous pouvez aussi importer les fichiers manuellement si Flyway est désactivé.

Ordre d'importation recommandé (via psql) :

```
# Core Schema
psql -U votre_user -d power_esta -f backend/src/main/resources/db/migration/V1_
psql -U votre_user -d power_esta -f backend/src/main/resources/db/migration/V2_

# Extensions et Corrections (Racine du projet)
psql -U votre_user -d power_esta -f tables_sigb_manquantes.sql
psql -U votre_user -d power_esta -f all_permissions_complete.sql
```



3. Configuration du Backend (Spring Boot)

1. Accédez au dossier `backend/` : `cd backend`

2. Configurez vos variables d'environnement dans le fichier `.env` :

```
DB_URL=jdbc:postgresql://localhost:5432/power_esta
DB_USERNAME=votre_utilisateur_postgres
DB_PASSWORD=votre_mot_de_passe
JWT_SECRET=UneCleTresLongueEtSecreteDeMinimum32Caracteres

# Configuration Mail (Facultatif)
MAIL_USERNAME=votre_email@gmail.com
MAIL_PASSWORD=votre_mot_de_passe_application
```

3. Lancer le serveur :

```
./mvnw spring-boot:run
```

Le serveur sera accessible sur : `http://localhost:8080`

4. Configuration du Frontend (React)

1. Accédez au dossier `bibliotheque-app/` : `cd bibliotheque-app`

2. Installez les dépendances :

```
npm install
```

3. Lancez l'application :

```
npm run dev
```

L'application sera accessible sur : `http://localhost:5173`

C. Comptes de test prêts à l'emploi

Rôle	Email	Mot de passe
Administrateur	elvisouattara16@gmail.com	Watta561001@#@#

STRUCTURE DU PROJET

- `/backend` : API REST Spring Boot (Java).
- `/bibliotheque-app` : Interface utilisateur React + Vite + Tailwind CSS.
- `/*.sql` : Scripts de maintenance et d'initialisation de la base de données.

IV. Bilan du projet

Cette section offre une évaluation honnête et structurée des réalisations, des défis et des apprentissages du projet.

A. Ce qui a été Réalisé

Le projet a permis de livrer un Système Intégré de Gestion de Bibliothécaire fonctionnel, incluant les modules clés suivants :

- **Authentification et Sécurité** : Implémentation complète de l'authentification JWT, du 2FA (TOTP), et d'un système de gestion de rôles et permissions (RBAC) robuste.
- **Gestion des Utilisateurs** : Inscription, vérification par email, gestion des profils et des accès.
- **Gestion du Catalogue de Livres** : Fonctionnalités CRUD complètes pour les livres et leurs exemplaires, avec recherche multicritère.
- **Gestion des Emprunts et Réservations** : Automatisation des processus de prêt, de retour, de renouvellement, de calcul des pénalités et de gestion des files d'attente de réservation.
- **Système de Notifications** : Envoi automatisé de rappels de retard, de notifications de disponibilité et de confirmations via email, grâce à un Scheduler quotidien.
- **Interface Utilisateur** : Application web responsive et application desktop multiplateforme (via Tauri) offrant une expérience utilisateur cohérente et intuitive.

B. Les fonctionnalités non implémentées

Malgré l'atteinte des objectifs principaux, certaines fonctionnalités envisagées n'ont pas pu être intégrées dans cette version initiale, principalement en raison de contraintes de temps et de complexité technique :

- **Paiement Mobile Money Intégré** tels que cinetpay: L'intégration directe avec des plateformes de paiement mobile (Orange Money, Moov Money) pour les cotisations et

amendes n'a pas été finalisée. Le système prévoit un calcul des pénalités, mais le règlement reste manuel.

- Notifications SMS : Bien que le système de notification soit conçu pour supporter plusieurs canaux, l'intégration avec un fournisseur de SMS n'a pas été mise en œuvre.
- Application Tauri Complète (Synchronisation Hors Ligne Avancée) : La version desktop via Tauri est fonctionnelle, mais une synchronisation hors ligne plus robuste et complexe, permettant une utilisation prolongée sans connexion internet avec une résolution automatique des conflits, n'a pas été entièrement développée.

C. Difficultés Rencontrées et Solutions Apportées

Le parcours de développement a été jalonné de plusieurs défis techniques :

- Gestion des États Concurrents : Assurer l'intégrité des données lors de multiples opérations simultanées (ex: deux utilisateurs réservant le même livre). Solution : Utilisation de verrous au niveau de la base de données et de transactions atomiques pour garantir la cohérence.
- Configuration de Tauri avec React : L'intégration du frontend React dans l'environnement Tauri a nécessité une configuration minutieuse, notamment pour la gestion des chemins et des ressources statiques. Solution : Référence à la documentation officielle de Tauri et ajustements des scripts de build.
- Mise en Place du TOTP pour le 2FA : L'implémentation du TOTP a requis une compréhension approfondie des algorithmes de génération de codes et de leur synchronisation temporelle. Solution : Utilisation de bibliothèques Java existantes pour le TOTP et tests rigoureux des scénarios d'activation et de vérification.

D. Expériences acquises

Ce projet a été une expérience d'apprentissage enrichissante, tant sur le plan technique que méthodologique :

- Compétences Techniques : Maîtrise approfondie de l'écosystème Spring Boot (Security, Data JPA), développement d'interfaces réactives avec React, et déploiement multiplateforme avec Tauri. Compréhension des enjeux de sécurité (JWT, 2FA) et d'optimisation des bases de données.

- **Compétences Méthodologiques** : Importance d'une modélisation UML rigoureuse, gestion de projet agile, travail en équipe, et capacité à résoudre des problèmes complexes de manière itérative.

CONCLUSION GENERALE ET PERSPECTIVES

I. Bilan du projet

Le projet de conception et de réalisation d'un Système Intégré de Gestion de Bibliothèque (SIGB) pour l'ESTA a été mené à bien, aboutissant à une solution robuste, sécurisée et conviviale. Les objectifs initiaux, à savoir l'automatisation de la gestion des emprunts, la sécurisation des accès, la notification en temps réel des utilisateurs et la fourniture d'outils de pilotage précis aux bibliothécaires, ont été atteints avec succès. Le système développé, basé sur une architecture moderne combinant Spring Boot pour le backend et React/Tauri pour le frontend et l'application desktop, offre une grande flexibilité et une maintenabilité accrue. L'intégration de fonctionnalités avancées telles que l'authentification à deux facteurs (2FA), la gestion des rôles et permissions (RBAC), et un système de notifications automatisé, positionne ce SIGB comme une solution de pointe pour la gestion des ressources documentaires. La modélisation UML exhaustive a servi de fondation solide, garantissant une compréhension approfondie des processus métier et une implémentation fidèle aux spécifications.

II. Limites du projet

Bien que le SIGB réponde aux besoins fondamentaux de la bibliothèque de l'ESTA, certaines limites ont été identifiées ou n'ont pas pu être entièrement explorées dans le cadre de ce projet initial :

- **Intégration avec des Systèmes Externes** : Le système actuel est autonome. Une intégration avec des systèmes universitaires existants (gestion des étudiants, systèmes de paiement centralisés) n'a pas été implémentée, ce qui pourrait simplifier d'avantage les processus administratifs.

- **Gestion Avancée des Médias** : La gestion se concentre principalement sur les livres physiques. L'intégration de ressources numériques (e-books, revues en ligne) et de leurs spécificités de prêt et d'accès n'a pas été abordée en profondeur.
- **Personnalisation Poussée** : Bien que l'interface soit responsive, les options de personnalisation pour les utilisateurs finaux (thèmes, agencement des tableaux de bord) sont limitées.
- **Analyse Prédictive** : Les statistiques actuelles sont descriptives. L'ajout de capacités d'analyse prédictive (par exemple, prévision des besoins en acquisitions basée sur les tendances d'emprunt) n'a pas été intégré.

III. Perspectives d'Evolution

Le SIGB développé constitue une base solide pour de futures améliorations et extensions. Plusieurs pistes d'évolution sont envisageables pour enrichir le système et répondre aux besoins futurs, en s'appuyant sur les retours d'expérience et les avancées technologiques :

- **Intégration d'un Moteur de Recommandation** : Développer un système de recommandation intelligent capable de suggérer des livres aux utilisateurs en fonction de leur historique d'emprunt, de leurs préférences et des tendances de lecture, enrichissant ainsi l'expérience utilisateur.
- **Paiement Mobile Money Intégré** : Mettre en place des passerelles de paiement directes avec les services de Mobile Money locaux (Orange Money, Moov Money) pour faciliter le règlement des cotisations et des amendes en ligne, offrant une solution de paiement plus accessible et rapide.
- **Application Mobile Native (Android/iOS)** : En complément de l'application desktop, développer une application mobile native via React Native. Cela permettrait d'offrir une expérience utilisateur optimisée sur smartphones et tablettes, avec des fonctionnalités spécifiques comme la numérisation de codes-barres pour l'inventaire ou la consultation rapide des informations.
- **Synchronisation Hors Ligne Robuste pour Tauri** : Améliorer la version desktop Tauri pour une synchronisation hors ligne plus robuste, permettant une utilisation prolongée

sans connexion internet et une résolution automatique des conflits lors de la reconnexion, garantissant ainsi la continuité du service.

- **Module de Rapports PDF Automatiques** : Développer un module capable de générer automatiquement des rapports d'activité mensuels ou trimestriels en format PDF pour la direction, incluant des statistiques clés sur les emprunts, les retards et l'utilisation des ressources.
- **Intégration de Codes-barres/QR Codes Physiques** : Mettre en place un système de lecture de codes-barres ou QR codes physiques sur les exemplaires pour automatiser et accélérer les processus de retour en self-service, réduisant la charge de travail des bibliothécaires.
- **Tableau de Bord Analytique Avancé** : Développer des tableaux de bord interactifs et personnalisables pour les administrateurs, offrant des analyses plus fines et des outils de Business Intelligence avec des graphiques en temps réel pour l'aide à la décision stratégique.
- **Module de Gestion des Ressources Numériques** : Développer un module dédié à l'intégration et à la gestion des e-books, revues scientifiques et autres ressources numériques, incluant la gestion des licences et des accès.
- **Intégration avec l'Environnement Universitaire** : Mettre en place des API pour synchroniser les données utilisateurs avec le système d'information de l'université, et intégrer des passerelles de paiement pour les cotisations et amendes.

En conclusion, ce projet a démontré la faisabilité et l'efficacité d'un SIGB moderne pour l'ESTA. Les fondations techniques et fonctionnelles sont posées, ouvrant la voie à un développement continu et à l'adaptation aux évolutions technologiques et aux besoins des utilisateurs. Le système est prêt à transformer la gestion de la bibliothèque, la rendant plus efficace, sécurisée et centrée sur l'utilisateur.